TUGAS PRATIKUM BLOKPROSEDURAL

NAMA KELOMPOK :

1.AULIA IK 2411008

2.IKRIMANSA IK2411021

3.NADYA PRATIWI RISWANTO IK2411049

Tujuan Praktikum

- Memahami pembuatan dan penggunaan *stored procedure* di MySQL.
- Menerapkan variabel dan percabangan (IF) dalam database.
- Mengolah dan menampilkan data secara otomatis.
- Melakukan validasi dan analisis data akademik sederhana.

Deskripsi Studi Kasus

Studi kasus ini membahas pengolahan data akademik mahasiswa menggunakan *stored procedure* di MySQL. Data seperti nama, NIM, semester, SKS, IPK, dan status pembayaran digunakan untuk menghasilkan informasi, melakukan validasi kelayakan pengambilan KRS, menentukan beban studi, serta menilai performa akademik. Selain itu, sistem juga dapat membandingkan dua mahasiswa untuk menentukan siapa yang memiliki performa lebih baik berdasarkan IPK dan SKS.

Dasar teori singkat tentang blok prosedural

Blok prosedural adalah kumpulan perintah SQL yang disusun dalam satu kesatuan logika dan dijalankan sebagai satu proses, biasanya dalam bentuk *stored procedure*. Blok ini diawali dengan `BEGIN` dan diakhiri dengan `END`, serta dapat berisi deklarasi variabel (`DECLARE`), penugasan nilai (`SET`), dan struktur kontrol seperti `IF`, `ELSE`, dan `ELSEIF`.

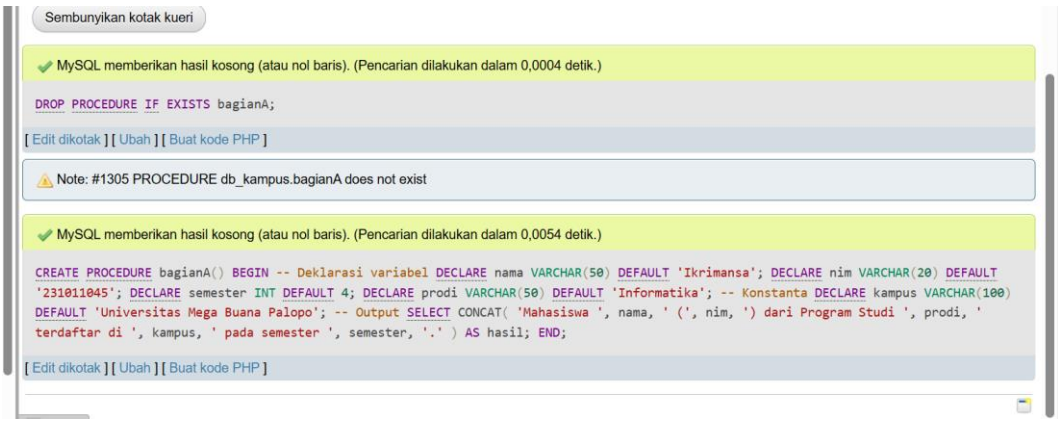
Dengan blok prosedural, proses pengolahan data menjadi lebih terstruktur, efisien, dan dapat digunakan kembali. Konsep ini memungkinkan database tidak hanya menyimpan data, tetapi juga menjalankan logika seperti pada bahasa pemrograman.

Kode Program

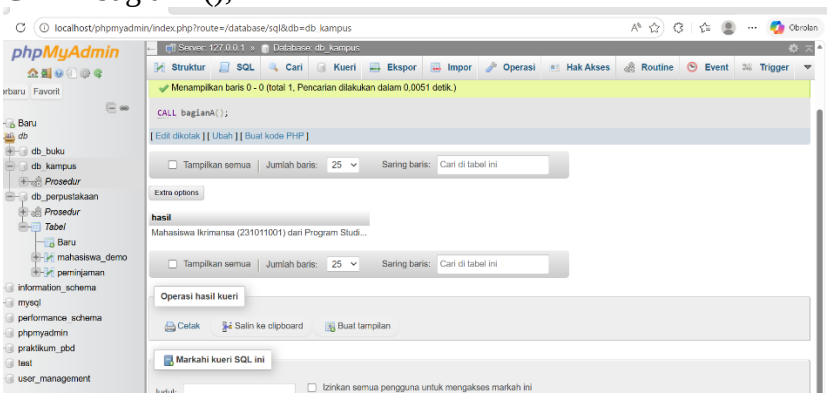
CALL bagianA();

```
DELIMITER //


CREATE PROCEDURE bagianA()

BEGIN

    DECLARE nama VARCHAR(50) DEFAULT 'Fazri';
```

```
    DECLARE nim VARCHAR(20) DEFAULT '231011001';

    DECLARE semester INT DEFAULT 4;

    DECLARE prodi VARCHAR(50) DEFAULT 'Informatika';

    DECLARE kampus VARCHAR(100) DEFAULT 'Universitas Mega Buana
Palopo';



    SELECT CONCAT(

        'Mahasiswa ', nama, ' (', nim, ') dari Program Studi ', prodi,

        ' terdaftar di ', kampus, ' pada semester ', semester, '.'

    ) AS hasil;

END //



DELIMITER ;
```



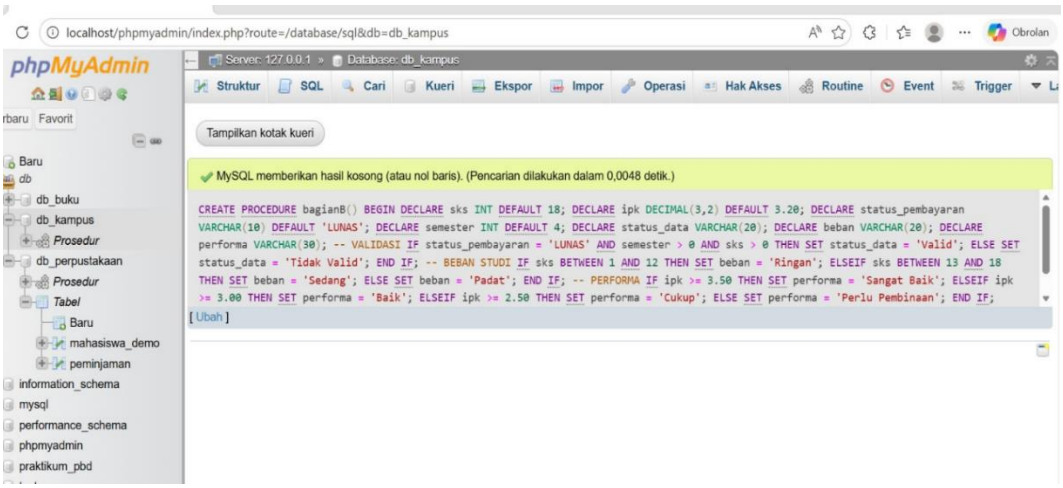
CALL bagianA();



```
CALL bagianB();
DELIMITER //

CREATE PROCEDURE bagianB()
BEGIN
    DECLARE sks INT DEFAULT 18;
    DECLARE ipk DECIMAL(3,2) DEFAULT 3.20;
    DECLARE status_pembayaran VARCHAR(10) DEFAULT 'LUNAS';
    DECLARE semester INT DEFAULT 4;

    DECLARE status_data VARCHAR(20);
    DECLARE beban VARCHAR(20);
    DECLARE performa VARCHAR(30);

    -- VALIDASI
    IF status_pembayaran = 'LUNAS' AND semester > 0 AND sks > 0 THEN
        SET status_data = 'Valid';
    ELSE
        SET status_data = 'Tidak Valid';
    END IF;

    -- BEBAN STUDI
```

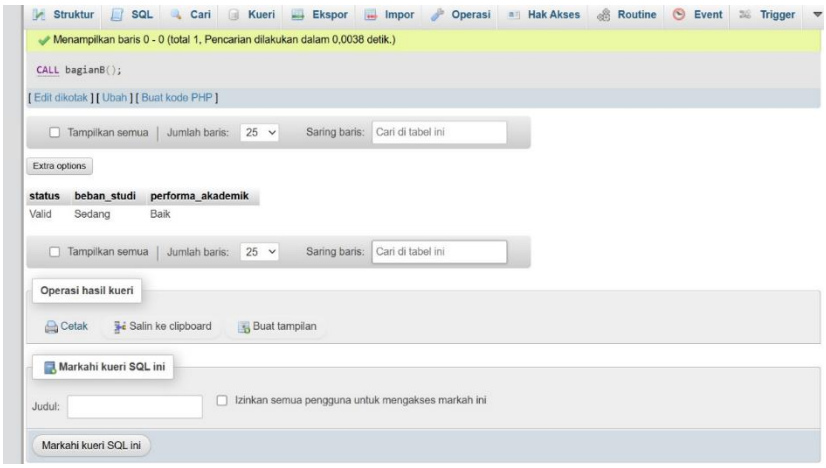
```
    IF sks BETWEEN 1 AND 12 THEN
        SET beban = 'Ringan';
    ELSEIF sks BETWEEN 13 AND 18 THEN
        SET beban = 'Sedang';
    ELSE
        SET beban = 'Padat';
    END IF;

    -- PERFORMA
    IF ipk >= 3.50 THEN
        SET performa = 'Sangat Baik';
    ELSEIF ipk >= 3.00 THEN
        SET performa = 'Baik';
    ELSEIF ipk >= 2.50 THEN
        SET performa = 'Cukup';
    ELSE
        SET performa = 'Perlu Pembinaan';
    END IF;

    SELECT
        status_data AS status,
        beban AS beban_studi,
        performa AS performa_akademik;
END //
```

DELIMITER ;



CALL bagianB();

Untuk



CALL bagianC();

DELIMITER //

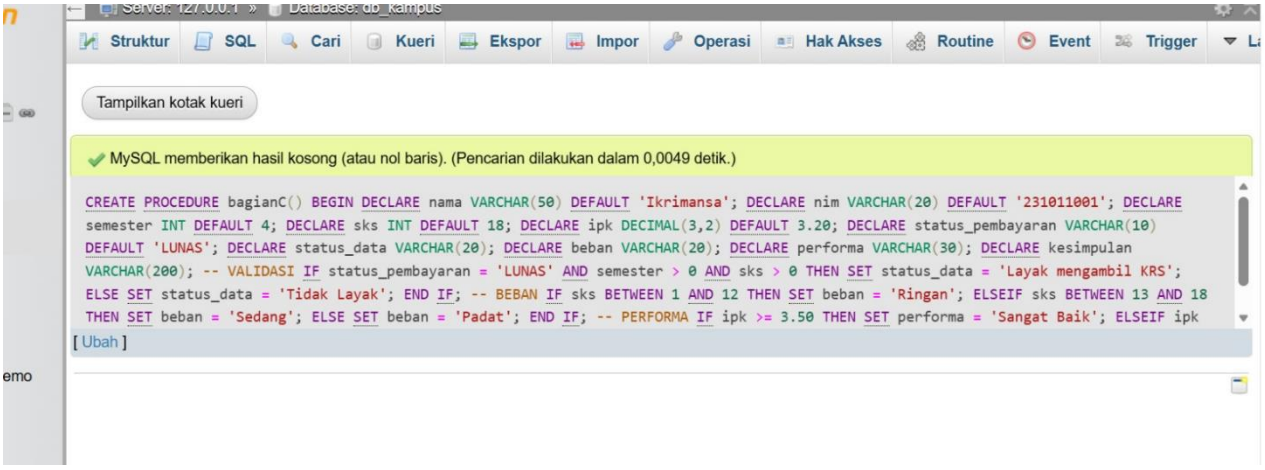
CREATE PROCEDURE bagianC()

BEGIN

    DECLARE nama VARCHAR(50) DEFAULT 'Fazri';

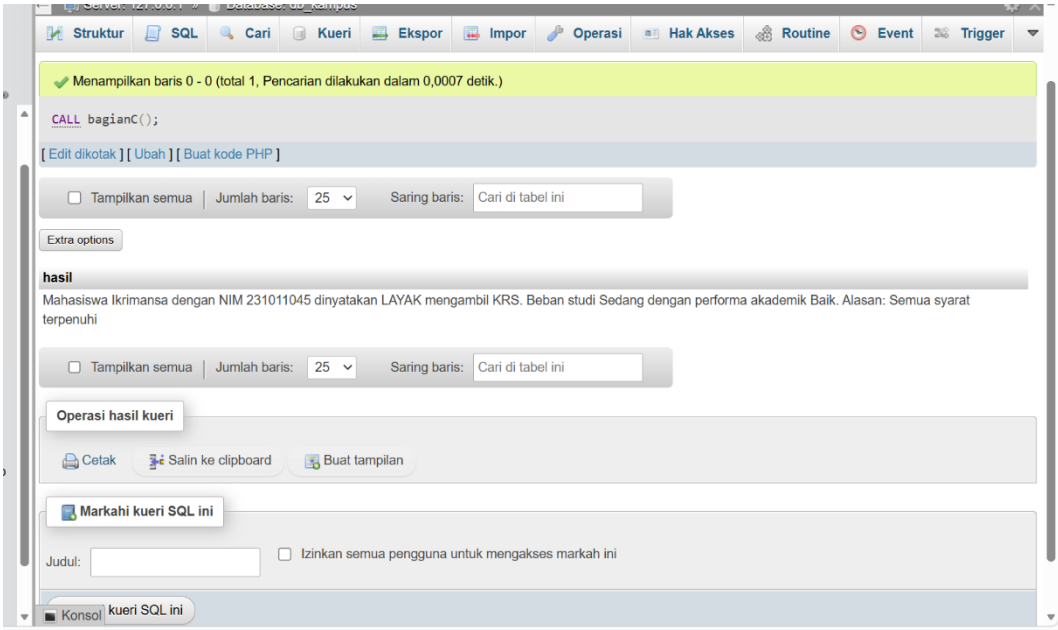
```
DECLARE nim VARCHAR(20) DEFAULT '231011001';
DECLARE semester INT DEFAULT 4;
DECLARE sks INT DEFAULT 18;
DECLARE ipk DECIMAL(3,2) DEFAULT 3.20;
DECLARE status_pembayaran VARCHAR(10) DEFAULT 'LUNAS';

DECLARE status_data VARCHAR(20);
DECLARE beban VARCHAR(20);
DECLARE performa VARCHAR(30);
DECLARE kesimpulan VARCHAR(200);

-- VALIDASI
IF status_pembayaran = 'LUNAS' AND semester > 0 AND sks > 0 THEN
   SET status_data = 'Layak mengambil KRS';
ELSE
   SET status_data = 'Tidak Layak';
END IF;

-- BEBAN
IF sks BETWEEN 1 AND 12 THEN
   SET beban = 'Ringan';
ELSEIF sks BETWEEN 13 AND 18 THEN
   SET beban = 'Sedang';
ELSE
```

```
        SET beban = 'Padat';
    END IF;

    -- PERFORMA
    IF ipk >= 3.50 THEN
        SET performa = 'Sangat Baik';
    ELSEIF ipk >= 3.00 THEN
        SET performa = 'Baik';
    ELSEIF ipk >= 2.50 THEN
        SET performa = 'Cukup';
    ELSE
        SET performa = 'Perlu Pembinaan';
    END IF;

    SET kesimpulan = CONCAT(
        'Mahasiswa ', nama, ' dengan NIM ', nim, ' dinyatakan ', status_data,
        '. Beban studi ', beban, ' dengan performa ', performa, '.'
    );

    SELECT kesimpulan;
END //

DELIMITER ;
```



CALL bagianC();

Untuk



CALL bagianD();

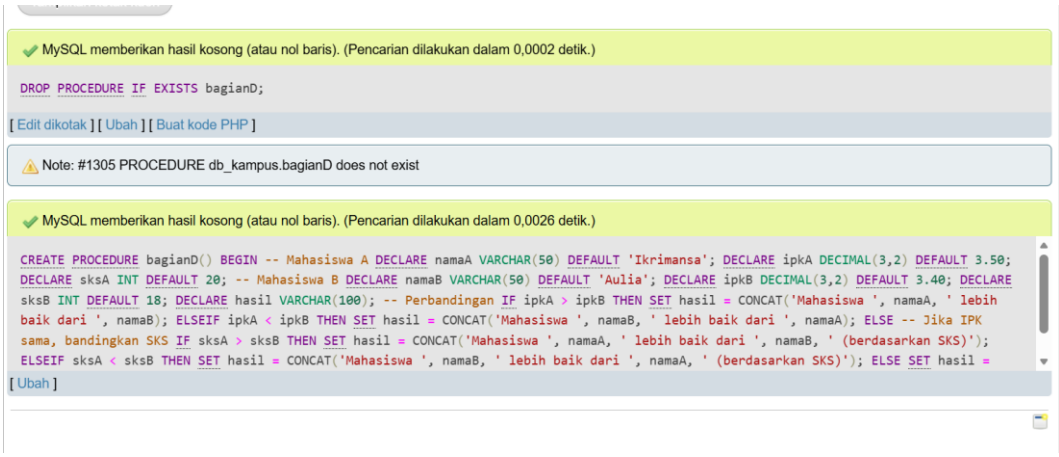
DELIMITER //

CREATE PROCEDURE bagianD()

```
BEGIN
    DECLARE nama1 VARCHAR(50) DEFAULT 'Fazri';
    DECLARE ipk1 DECIMAL(3,2) DEFAULT 3.50;
    DECLARE sks1 INT DEFAULT 20;

    DECLARE nama2 VARCHAR(50) DEFAULT 'Ikri';
    DECLARE ipk2 DECIMAL(3,2) DEFAULT 3.40;
    DECLARE sks2 INT DEFAULT 22;

    DECLARE hasil VARCHAR(100);

    IF ipk1 > ipk2 THEN
        SET hasil = CONCAT(nama1, ' lebih baik dari ', nama2);
    ELSEIF ipk2 > ipk1 THEN
        SET hasil = CONCAT(nama2, ' lebih baik dari ', nama1);
    ELSE
        IF sks1 > sks2 THEN
            SET hasil = CONCAT(nama1, ' lebih baik dari ', nama2);
        ELSE
            SET hasil = CONCAT(nama2, ' lebih baik dari ', nama1);
        END IF;
    END IF;

    SELECT hasil;
```

END //

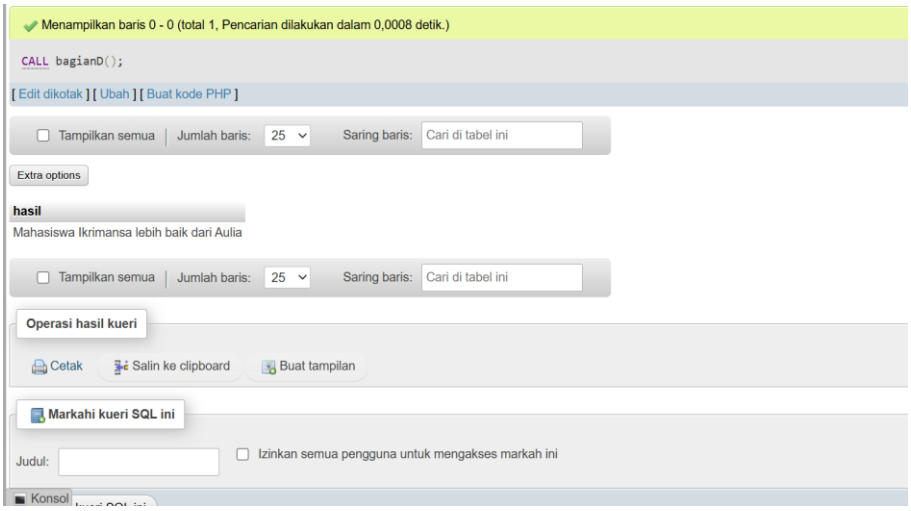
DELIMITER ;



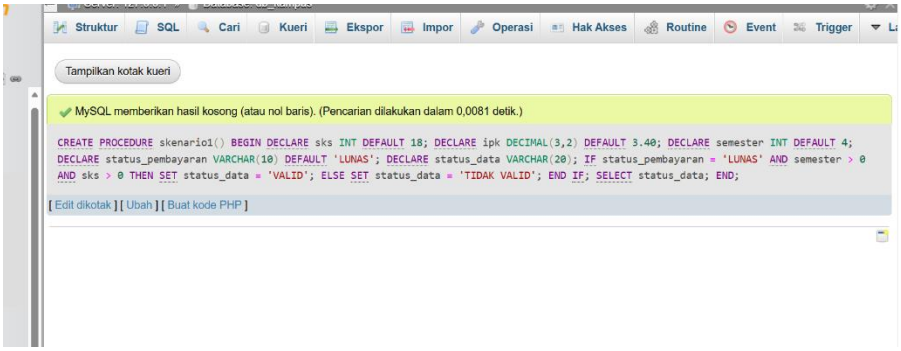
Lalu

CALL bagianD();

Untuk

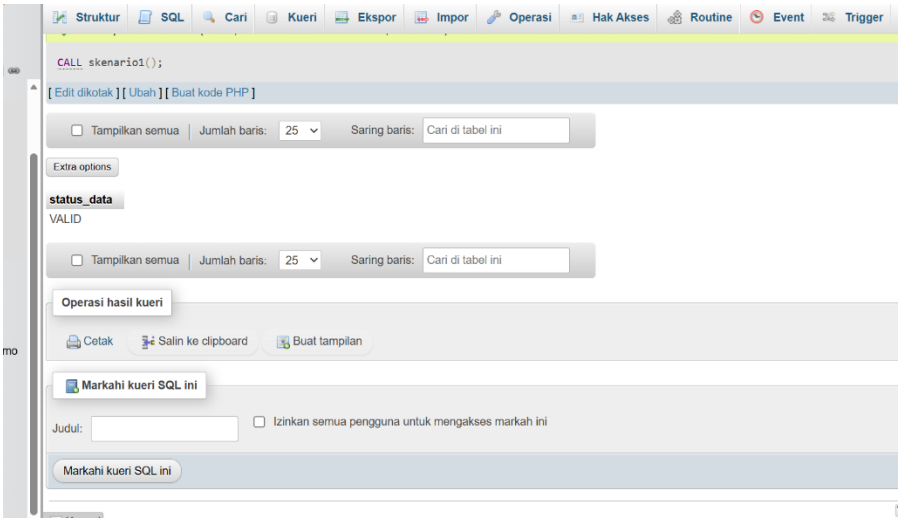


```
DELIMITER $$

CREATE PROCEDURE skenario1()
BEGIN
    DECLARE sks INT DEFAULT 18;
    DECLARE ipk DECIMAL(3,2) DEFAULT 3.40;
    DECLARE semester INT DEFAULT 4;
    DECLARE status_pembayaran VARCHAR(10) DEFAULT 'LUNAS';

    DECLARE status_data VARCHAR(20);

    IF status_pembayaran = 'LUNAS' AND semester > 0 AND sks > 0 THEN
        SET status_data = 'VALID';
    ELSE
        SET status_data = 'TIDAK VALID';
    END IF;

    SELECT status_data;
END$$

DELIMITER
```



CALL skenario1();

Untuk



DROP PROCEDURE IF EXISTS skenario2;

DELIMITER $$

CREATE PROCEDURE skenario2()

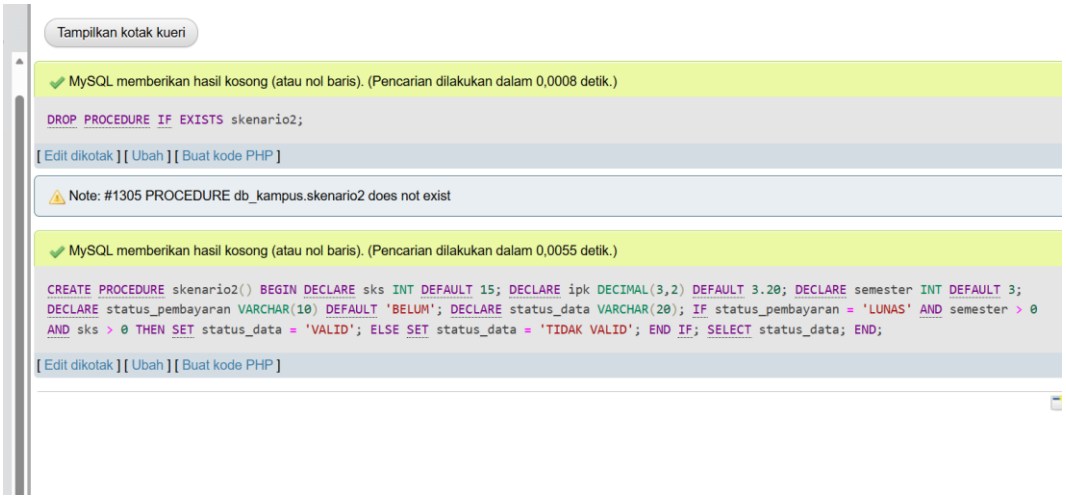
BEGIN

DECLARE sks INT DEFAULT 15;

DECLARE ipk DECIMAL(3,2) DEFAULT 3.20;

DECLARE semester INT DEFAULT 3;

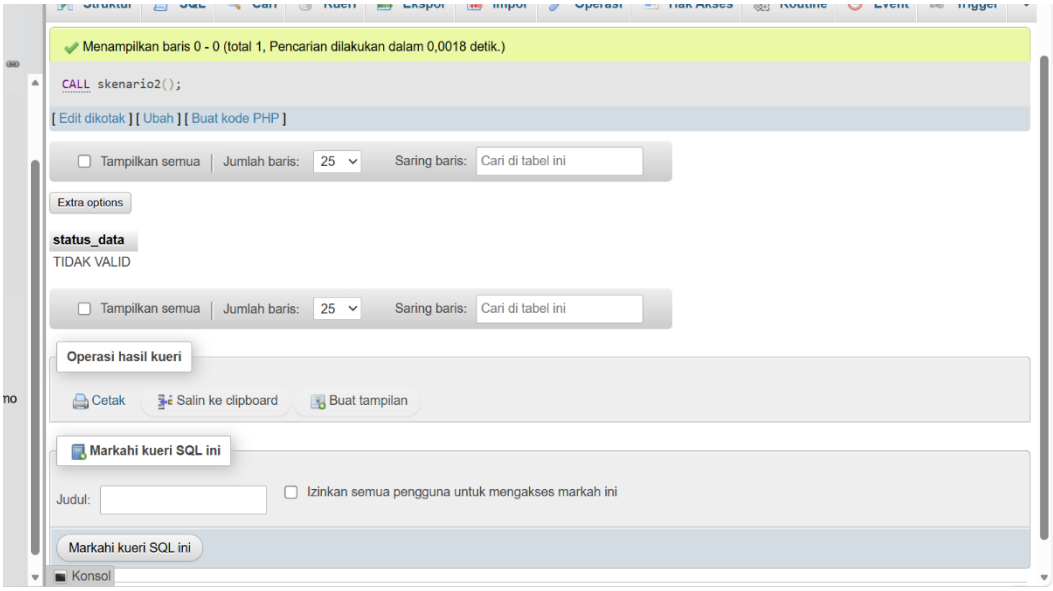
```
    DECLARE status_pembayaran VARCHAR(10) DEFAULT 'BELUM';

    DECLARE status_data VARCHAR(20);

    IF status_pembayaran = 'LUNAS' AND semester > 0 AND sks > 0 THEN
        SET status_data = 'VALID';
    ELSE
        SET status_data = 'TIDAK VALID';
    END IF;

    SELECT status_data;
END$$

DELIMITER ;
```



Lalu

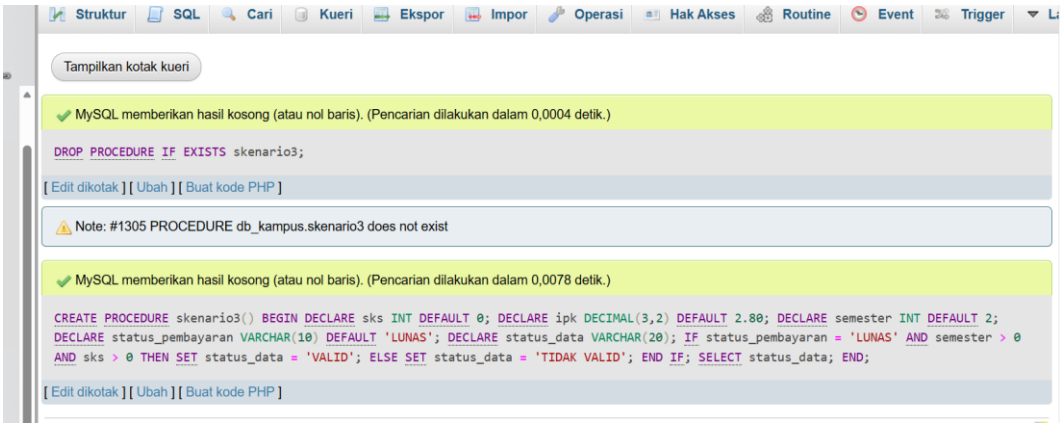
```
CALL skenario2();
```

Untuk




```
DROP PROCEDURE IF EXISTS skenario3;

DELIMITER $$

CREATE PROCEDURE skenario3()
BEGIN
    DECLARE sks INT DEFAULT 0;
    DECLARE ipk DECIMAL(3,2) DEFAULT 2.80;
    DECLARE semester INT DEFAULT 2;
    DECLARE status_pembayaran VARCHAR(10) DEFAULT 'LUNAS';

    DECLARE status_data VARCHAR(20);

    IF status_pembayaran = 'LUNAS' AND semester > 0 AND sks > 0 THEN
```

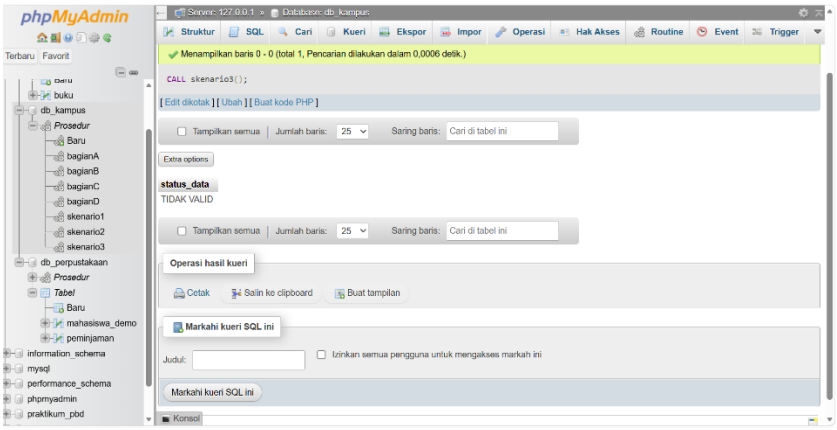
```
        SET status_data = 'VALID';
    ELSE
        SET status_data = 'TIDAK VALID';
    END IF;

    SELECT status_data;
END$$

DELIMITER ;
```



Lalu

CALL skenario3();

Untuk



Analisis singkat atas perbedaan hasil dari masing- masing skenario.

Skenario 1 (Valid)

Pada skenario ini, data mahasiswa dinyatakan VALID karena:

- Status pembayaran UKT = LUNAS
- Semester > 0
- Jumlah SKS > 0

Semua syarat terpenuhi sehingga:

- Mahasiswa layak mengambil KRS
- Beban studi dan performa akademik dapat dikategorikan dengan normal

Skenario 2 (Tidak Valid – Pembayaran)

Pada skenario ini, data dinyatakan TIDAK VALID karena:

- Status pembayaran = BELUM LUNAS

Meskipun:

- Semester valid
- SKS valid

Namun tetap:

- Mahasiswa tidak diperbolehkan mengambil KRS
- Faktor pembayaran menjadi penentu utama

Skenario 3 (Tidak Valid – SKS)

Pada skenario ini, data dinyatakan TIDAK VALID karena:

- Jumlah SKS = 0

Artinya:

- Mahasiswa tidak mengambil mata kuliah sama sekali
- Tidak memenuhi syarat minimal akademik

Sehingga:

- Tidak bisa diproses untuk KRS

Analisis Hasil

Hasil praktikum menunjukkan bahwa *stored procedure* dapat digunakan untuk menampilkan, memvalidasi, dan menganalisis data akademik secara otomatis. Setiap bagian berhasil menjalankan fungsinya, mulai dari menampilkan data, menentukan beban studi dan performa, hingga menghasilkan kesimpulan dan membandingkan mahasiswa. Hal ini membuktikan bahwa blok prosedural di MySQL bekerja efektif untuk pengolahan data.

Kesimpulan

Stored procedure dalam MySQL efektif digunakan untuk mengelola, memvalidasi, dan menganalisis data secara terstruktur. Dengan blok prosedural, proses pengolahan data menjadi lebih otomatis, efisien, dan mudah digunakan kembali.